

Ethical and Explainable Dynamic Pricing System for Ride-Sharing Platforms

Project Work Progress report submitted to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the

Award of the Degree

of

Master of Computer Applications

by

Parth Gaba

240970029

Under the guidance of

Pragya Jha

Assistant professor,
School Of Computer Engineering
M.I.T., Manipal - 576104

Ankur Bhardwaj

President,
Nucleocoders Technologies PVT LTD
NH-8,Celebration Gardens, Avenue Bouganvila,
Opp. IGIA, New Delhi

SCHOOL OF COMPUTER ENGINEERING MIT MANIPAL



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

April 2026

Table of Contents

1. Abstract.....	1
2. Introduction.....	2
2.1 Overview of the Project.....	2
2.2 Need for the System	2
3. Problem Statement	3
4. Objectives	3
4.1 Primary Objectives.....	3
4.2 Secondary Objectives.....	3
5. Scope.....	4
6. Methodology.....	4
6.1 Process Model.....	4
6.2 Development Approach.....	6
7. System Architecture.....	6
7.1 Overview Of Architecture.....	6
7.2 Layered Architecture.....	7
7.3 Data Flow.....	7
8. Work Completed.....	8
9. Challenges Faced.....	9
9.1 Technical Challenges.....	9
9.2 Integration Challenges.....	9
10. Future Work.....	10
10.1 Functional Enhancements.....	10
10.2 Performance Improvements.....	10
10.3 Scalablity Enhancements.....	10
11. Conclusion.....	11
12. References.....	11

1. Abstract

AurumRide is a full-stack ride-booking platform designed to deliver dynamic fare prediction with fairness, transparency, and production readiness. The system is built using a modern Next.js frontend (TypeScript, Tailwind CSS) integrated with a FastAPI backend, enabling scalable user interaction and robust API-driven operations. It includes a map-based booking interface, trip management features, and role-based dashboards for users and drivers.

The core contribution of the project is an ethical and explainable dynamic pricing engine. Fare prediction is powered by an XGBoost regression model developed through a reproducible machine learning pipeline, incorporating key contextual factors such as demand–supply conditions, traffic, time, and weather. To prevent unfair pricing behavior, backend-enforced ethical guardrails restrict surge pricing in sensitive scenarios, while final fares are recomputed server-side at booking time to ensure integrity and consistency.

To enhance transparency, the system integrates SHAP-based Explainable AI, providing an “Explain Price” feature that reveals the contribution of individual factors to each fare prediction. Additionally, pricing context and ethical decisions are stored as audit data to support traceability and analysis. The platform is further extended with real-time ride simulation, driver-specific data isolation, and improved authentication flows, enabling a realistic and production-oriented user experience.

Overall, AurumRide demonstrates how machine learning-driven dynamic pricing can be made accurate, transparent, and ethically accountable within a deployable full-stack application.

2. Introduction

2.1 Overview of the existing project

Ride-sharing platforms have significantly transformed urban transportation by enabling on-demand mobility through digital applications. Services such as Uber and Ola utilize dynamic pricing mechanisms to balance rider demand and driver availability in real time. These systems consider multiple contextual factors, including demand–supply ratio, traffic conditions, time of day, and weather, to generate fare estimates dynamically.

While dynamic pricing improves operational efficiency and resource allocation, most existing systems function as black-box models, offering limited visibility into how fares are computed. Users are typically presented with a final price without understanding the contributing factors, which can reduce trust and transparency in the platform.

AurumRide is designed as a full-stack ride-booking platform that integrates modern web technologies with machine learning to address these challenges. Built using a Next.js frontend and FastAPI backend, the system provides a complete booking workflow, including map-based ride selection, trip management, and role-based dashboards for users and drivers.

At its core, AurumRide incorporates an ethical and explainable dynamic pricing engine powered by an XGBoost regression model. The system enhances traditional pricing approaches by integrating fairness constraints, explainability mechanisms, and backend validation to ensure that fare computation is not only accurate but also transparent and accountable.

2.2 Need for the System

Despite the widespread adoption of dynamic pricing in ride-sharing platforms, several limitations highlight the need for a more transparent and ethically governed system. One major issue is the lack of explainability in pricing decisions. Users often experience sudden fare increases without clarity on the underlying reasons, leading to dissatisfaction and reduced trust in the platform.

Another critical concern is the absence of ethical constraints in conventional surge pricing models. During high-demand scenarios such as heavy rainfall, peak traffic hours, or emergency situations, pricing algorithms may generate excessively high fares. Without proper safeguards, such outcomes can be perceived as unfair and socially insensitive.

Additionally, existing systems often lack accountability mechanisms to trace how a particular fare was computed. This makes it difficult for both users and system administrators to audit pricing decisions or identify anomalies in the pricing logic.

AurumRide addresses these challenges by introducing a system that combines machine learning-based pricing with ethical guardrails and explainable AI. By providing transparency through SHAP-based explanations, enforcing fairness through backend constraints, and maintaining audit-level data for traceability, the system aims to improve user trust, fairness, and reliability in dynamic pricing.

3. Problem Statement

Ride-sharing platforms rely on dynamic pricing mechanisms to balance rider demand and driver availability in real time. While this approach improves system efficiency and resource utilization, most existing pricing systems operate as black-box models, providing limited transparency into how fares are calculated. Users are typically presented with a final price without any explanation of the contributing factors, which can lead to confusion, dissatisfaction, and reduced trust in the platform.

Another significant issue is the absence of ethical safeguards in conventional surge pricing models. During high-demand or sensitive conditions such as heavy rainfall, peak traffic hours, or emergency situations, pricing algorithms may generate excessively high fares. Such outcomes can be perceived as unfair or exploitative, as the system prioritizes demand-supply optimization without considering social and ethical implications.

Additionally, current systems often lack accountability and traceability in pricing decisions. There is minimal provision to audit how a particular fare was derived or to analyze the influence of individual factors. This limits both user understanding and administrative control over pricing behavior.

Therefore, there is a need to design a dynamic pricing system that is not only accurate and efficient but also transparent, explainable, and ethically governed. The problem addressed in this project is the development of a full-stack ride-booking platform that integrates machine learning-based fare prediction with ethical surge constraints and explainable AI, ensuring that pricing decisions are understandable, fair, and accountable.

4. Objective

4.1 Primary Objectives

The primary objectives of the AurumRide project are:

1. To develop a machine learning-based dynamic fare prediction system using regression techniques, with emphasis on the XGBoost algorithm.
2. To design and implement an ethical pricing framework that incorporates guardrails to prevent excessive surge pricing in sensitive conditions.
3. To integrate Explainable AI using SHAP in order to provide transparent and interpretable fare predictions.
4. To ensure pricing integrity by implementing server-side fare computation and booking-time revalidation.
5. To build a full-stack ride-booking platform that combines frontend interaction, backend processing, and ML-based decision-making in a unified system.

4.2 Secondary Objectives

The secondary objectives that support the primary goals include:

1. To incorporate contextual factors such as demand–supply conditions, traffic, time, and weather into the pricing model.
2. To design a user-friendly interface using Next.js that supports booking, trip tracking, and price explanation features.
3. To implement role-based access control (RBAC) for users and drivers to ensure secure and

structured system access.

4. To provide audit-level data storage for pricing decisions to enable traceability and post-analysis.
5. To simulate real-time ride experience using map-based interaction and driver movement visualization.
6. To establish a reproducible machine learning pipeline for model training, evaluation, and deployment.

5. Scope

The scope of the AurumRide project focuses on the design and implementation of an ethical and explainable dynamic pricing system within a ride-booking platform. The system aims to generate accurate fare predictions using machine learning while ensuring fairness, transparency, and accountability in pricing decisions.

The project includes the development of a full-stack application consisting of a Next.js frontend and a FastAPI backend. The frontend provides a map-based booking interface, trip management features, and role-based dashboards for users and drivers. The backend handles authentication, booking workflows, pricing logic, and secure API operations.

Within the scope of pricing, the system utilizes contextual variables such as demand–supply conditions, traffic levels, time of day, and weather data to compute fare estimates. An XGBoost regression model is employed for prediction, supported by an ethical pricing layer that applies surge constraints in sensitive scenarios. Additionally, SHAP-based explainability is integrated to provide transparency by showing how different features contribute to the final fare.

The system also includes supporting features such as role-based access control, driver-specific data isolation, booking history management, rating and feedback mechanisms, and real-time ride simulation for demonstration purposes. Pricing-related audit data is stored to enable traceability and analysis of fare decisions.

However, the project is limited to a prototype-to-deploy academic implementation and does not include integration with real-world ride-hailing services, live dispatch systems, payment gateways, or large-scale distributed infrastructure. External data sources such as real-time emergency feeds or government systems are also not integrated. The system operates on simulated or controlled datasets and focuses primarily on demonstrating the feasibility and effectiveness of ethical and explainable dynamic pricing.

6. Methodology

6.1 Process Model

The development of AurumRide follows a modular and iterative full-stack machine learning workflow, where each component is designed, implemented, and validated in stages. The overall process integrates data-driven modeling with backend services and frontend interaction.

The methodology consists of the following steps:

1. Data Preparation

Historical and simulated ride data is collected and structured, including features such as distance, demand–supply ratio, traffic conditions, time, and weather context.

2. Data Preprocessing

The dataset is cleaned to handle missing values, inconsistencies, and outliers. Feature engineering is performed to transform raw inputs into meaningful variables suitable for model training.

3. Feature Analysis

Key pricing factors are analyzed to understand their influence on fare prediction. This helps in selecting relevant variables and improving model performance.

4. Model Development

A regression-based machine learning model is developed using XGBoost. The model is trained on prepared data and evaluated using metrics such as Mean Absolute Error (MAE) to ensure prediction accuracy.

5. Ethical Pricing Layer

An additional logic layer is implemented in the backend to enforce ethical constraints. Surge pricing is capped or adjusted during sensitive scenarios (e.g., heavy rain), ensuring fairness in fare computation.

6. Explainability Integration

SHAP (SHapley Additive Explanations) is integrated to generate feature-level contribution insights. This enables the system to explain how each factor influences the predicted fare.

7. Backend Integration

The trained model is deployed within FastAPI endpoints to provide real-time fare predictions. Fare values are recomputed server-side at booking time to maintain integrity.

8. Frontend Integration

The Next.js frontend consumes backend APIs to display fare estimates, booking workflows, and the “Explain Price” feature to users.

9. Testing and Validation

The system is validated through backend testing (pytest) and frontend checks (linting and build validation), ensuring stability and correctness.

6.2 Development Approach

The development of AurumRide follows a **modular, API-driven, and iterative approach**, enabling scalability and maintainability.

- **Full-Stack Architecture**
The system is divided into:
 - Frontend (Next.js) for user interaction
 - Backend (FastAPI) for business logic and APIs
 - ML layer for pricing intelligence
- **API-Driven Design**
All functionalities such as authentication, booking, pricing, and driver operations are exposed via REST APIs, ensuring clear separation between frontend and backend.
- **Role-Based Access Control (RBAC)**
The system enforces structured access for users and drivers, ensuring secure and controlled operations.
- **Iterative Enhancement**
The platform is continuously improved by adding features such as authentication flows, driver-specific data isolation, ride simulation, and rating systems.
- **Reproducible ML Pipeline**
Model training, evaluation, and deployment are structured to ensure consistency and reproducibility.
- **Scalability Considerations**
The architecture is designed to be extendable for future integration with production-level systems such as cloud deployment and real-time data pipelines.

7. System Architecture

7.1 Overview of Architecture

AurumRide follows a modular, layered full-stack architecture that integrates a modern web frontend, a scalable backend service layer, and a machine learning-based pricing engine. The system is designed using an API-driven approach, where all core functionalities such as authentication, booking, pricing, and driver management are handled through RESTful APIs.

The frontend is developed using Next.js and provides an interactive user interface for ride booking, trip management, and price explanation. The backend, built with FastAPI, handles business logic, user authentication, booking workflows, and pricing computations. The machine learning layer is integrated into the backend to provide real-time fare predictions along with explainability using SHAP.

This architecture ensures clear separation of concerns, making the system scalable, maintainable, and suitable for further extension into production environments.

7.2 Layered Architecture

1. Presentation Layer (Frontend)

- Built using Next.js (TypeScript, Tailwind CSS)
- Provides user interface for:
 - Ride booking via map interaction
 - Viewing trip history
 - Accessing “Explain Price” feature
 - Driver dashboards and profile management
- Communicates with backend using REST APIs

2. Application Layer (Backend)

- Developed using FastAPI
- Responsible for:
 - User authentication and role-based access control (RBAC)
 - Booking creation, management, and status updates
 - Driver profile handling and assignment.
 - API endpoints for pricing, bookings, and user operations.
 - Ensures secure and validated data exchange between frontend and ML components

3. Data & ML Layer

- Includes:
 - SQLAlchemy-based database (SQLite by default)
 - XGBoost regression model for fare prediction
 - SHAP for explainable AI
- Handles:
 - Storage of users, drivers, bookings, and audit data
 - Model inference for real-time fare prediction
 - Generation of feature-level explanation outputs
- Supports reproducible training and evaluation pipeline

7.3 Data Flow

The system follows a structured data flow from user interaction to final fare computation and booking:

1. User Input

The user selects pickup and drop locations via the map or manual input on the frontend.

2. Request to Backend

The frontend sends ride details (distance, time, location context, etc.) to the backend through API calls.

3. Pricing Computation

The backend processes the request and:

- Fetches contextual data (e.g., weather)
- Sends input to the ML model (XGBoost) for fare prediction

4. Ethical Guardrail Application

The predicted fare is adjusted using backend-defined ethical constraints to prevent excessive surge pricing.

5. **Explainability Generation**

SHAP is used to compute feature contributions, which are sent back as part of the response.

6. **Response to Frontend**

The backend returns:

- Final fare estimate
- Explanation data
- Contextual details

7. **User Interaction & Booking**

The user reviews the fare and explanation, then confirms the booking.

8. **Booking Validation & Storage**

The backend revalidates the fare, creates a booking entry, and stores audit data for traceability.

9. **Ride Simulation & Updates**

The frontend displays ride confirmation, driver details, and simulated real-time updates.

8. **Work Completed**

The AurumRide system has been developed as a full-stack ride-booking platform integrating machine learning-based pricing, ethical constraints, and explainability. The major components implemented are as follows:

8.1 Core System Implementation

- Developed a FastAPI backend supporting authentication, RBAC, drivers, bookings, and pricing services.
- Built a Next.js frontend with map-based booking, trip management, and role-based dashboards.
- Integrated Leaflet-based maps with client-side rendering for smooth interaction.
- Implemented a weather-aware dynamic pricing system using an XGBoost regression model.
- Enforced ethical guardrails to cap surge pricing in sensitive scenarios.
- Integrated SHAP-based Explain Price functionality for transparency.
- Implemented server-side fare revalidation and audit data persistence.

8.2 Authentication and Role Management

- Designed dedicated Login and Registration pages.
- Implemented role-based authentication (User and Admin/Driver).
- Enhanced UI with user identity display and improved error handling.

8.3 Driver-Centric Architecture

- Mapped each admin to a personal driver profile with editable details.
- Enforced strict data isolation where drivers can access only their assigned bookings.

8.4 Booking and User Features

- Implemented driver-specific booking views and controlled status updates.
- Added password management features (forgot and update password).

- Integrated reverse geocoding and manual address search for accurate pickup/drop locations.

8.5 Ride Experience Enhancements

- Developed an Uber-like confirmation system showing driver details, fare, ETA, and booking ID.
- Implemented live driver availability indicators using `/api/drivers/nearby`.
- Added real-time ride simulation with driver movement, route visualization, and status updates.

8.6 Additional Features and Improvements

- Extended driver model with vehicle number and rating.
- Implemented post-ride rating and feedback system.
- Fixed ETA inconsistencies and improved synchronization using periodic polling.
- Enforced role-based navigation and separation of user and driver workflows.
- Optimized performance for smooth and demo-ready interaction.

8.7 Validation

- Backend testing using pytest (successful execution).
- Frontend validation using `npm run lint` and `npm run build`.
- Verified end-to-end functionality across booking, pricing, and simulation workflows.

9. Challenges Faced

9.1 Technical Challenges

During the development of AurumRide, several core technical challenges were encountered:

- **Machine Learning Model Integration**

Integrating the XGBoost model into the backend required ensuring consistent feature input, proper preprocessing, and efficient real-time inference within API responses.

- **Explainability vs Performance Trade-off**

SHAP-based explainability introduced computational overhead, making it challenging to maintain low response times while providing detailed feature-level explanations.

- **Database Schema Evolution**

Adding new attributes such as driver ratings and booking feedback required handling schema updates carefully without disrupting existing data (SQLite migration handling).

- **State Management and Simulation Logic**

Implementing a stable ride simulation required handling state transitions correctly and preventing

repeated triggers due to frontend updates.

- **Map Rendering Constraints**

Integrating Leaflet with a server-side rendered framework (Next.js) required client-side dynamic loading to avoid runtime errors.

9.2 Integration Challenges

In addition to technical issues, several integration-related challenges were faced:

- **Frontend–Backend Communication**

Ensuring seamless API communication between Next.js and FastAPI required well-defined endpoints, consistent data formats, and proper error handling.

- **Data Consistency Across Layers**

Maintaining synchronization between frontend UI (ETA, booking status) and backend data required implementing polling mechanisms and update controls.

- **Role-Based Access Control Integration**

Enforcing RBAC across the system required tight coupling between authentication logic and API-level authorization to prevent unauthorized access.

- **Multi-Component Coordination**

Coordinating interactions between the frontend, backend, ML model, and database required careful orchestration to ensure smooth end-to-end workflows.

10. Future Work

10.1 Functional Enhancements

- Expand ethical pricing policies to cover multiple real-world scenarios beyond weather conditions. Develop advanced analytics dashboards with downloadable reports for administrative users.
- Integrate real-time external APIs (traffic, weather) for more accurate predictions.
- Add payment gateway integration and real ride completion workflows.
- Enhance user experience with features such as ride scheduling, notifications, and improved UI interactions.

10.2 Performance Improvements

- Optimize SHAP-based explainability computations to reduce response time during fare prediction.
- Introduce caching mechanisms for frequently accessed data such as pricing results and driver availability.
- Improve backend query efficiency and indexing for faster database operations.
- Optimize frontend rendering and API calls to enhance user experience and responsiveness

10.3 Scalability Enhancements

- Migrate from SQLite to a production-grade database such as PostgreSQL for better scalability and concurrency handling.
- Deploy the system on cloud infrastructure (e.g., AWS, Azure) to support real-world usage.
- Introduce containerization (Docker) and microservices-based architecture for independent scaling of components.
- Implement load balancing and API rate limiting to handle high user traffic efficiently.

11. Conclusion

AurumRide demonstrates that dynamic pricing in ride-sharing systems can be designed to be not only accurate but also transparent and ethically responsible. By integrating machine learning with fairness constraints and explainable AI, the project addresses key limitations of traditional black-box pricing models and improves user trust in fare computation.

The system successfully combines a Next.js frontend, FastAPI backend, and an XGBoost-based pricing engine into a cohesive full-stack application. Features such as SHAP-based price explanation, ethical surge guardrails, server-side fare validation, and role-based workflows contribute to a robust and production-oriented design. Additionally, enhancements like real-time ride simulation, driver-specific data isolation, and improved user experience make the platform more realistic and interactive.

Overall, the project demonstrates a complete machine learning lifecycle, from data preparation and model training to deployment and real-time inference within an application environment. While further improvements in scalability, performance, and testing are planned, the current system provides a strong foundation for building trustworthy and accountable dynamic pricing solutions in modern mobility platforms.

12. References

1. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proc. ACM SIGKDD, 2016.
2. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS, 2017.
3. J. Pearl, The Book of Why: The New Science of Cause and Effect. Basic Books, 2018.
4. FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com/>
5. Next.js Documentation. [Online]. Available: <https://nextjs.org/docs>